ProofLoop Global: Final Product Specification v1.2

Oct 27, 2026 · @RUDRANARAYAN RAY

Assessment of v1.1

v1.1 is a strong and unusually honest spec, but it describes roughly three products (a marketplace, a negotiation tool and a payment-evidence layer) and you have 16 days (27 Oct to 12 Nov). Keep the idea, and build one vertical slice around its most original part: an unbroken evidence chain from the approved agreement version to the verified PayPal payment state.

What is already strong

- **Truthfulness rules.** Never claiming escrow, bank credit, live FX or fees that PayPal did not return is a real differentiator and fits PayPal's own documentation.
- **Immutable, hash-approved agreement versions.** Both sides approve the same version, and a counteroffer creates a new one. This is simple to build and easy to demonstrate.
- **Reconcile-before-retry payment recovery.** Treating a timeout as UNKNOWN, not failed, is correct engineering and the strongest 'Technological Implementation' story in the spec.
- **Clear approval model.** AI drafts, humans approve, and the backend state machine decides payment status. This also fits the agentic-commerce theme.
- **Honest limits.** The crypto section correctly keeps ETH out of the MVP.

Gaps found and fixed in v1.2

#	Gap in v1.1	Why it matters	Fix in v1.2
1	Scope: marketplace discovery, reputation, crypto, route comparison, 24 features, 16 days	A half-working everything loses to one flawless flow	Scope tiers and a cut list (Scope tiers section)

#	Gap in v1.1	Why it matters	Fix in v1.2
2	No funding or payee model: the spec never says who receives the captured money	PayPal assumes the API caller's account when no payee is given, and third-party payee and platform fees sit in limited-release commerce-platform docs	Platform sandbox merchant receives funds; freelancer payout is a recorded, simulated ledger entry (Payments section)
3	Payment states disagree across sections 7.2, 16.1 and 16.3; approval reversed and needs support are missing from the state list	The state machine is the core of the product	One state machine with one UI mapping
4	Idempotency over-trusted	PayPal's Orders capture reference says keys are stored for 6 hours, while a general page says up to 45 days	Idempotency is a short-window safety net; reconciling with GET order is the primary mechanism
5	Webhook testing plan missing	Simulator events cannot be verified by posting back to PayPal, and events fire only for the app that processed the action	Test with real sandbox transactions; public HTTPS URL via tunnel or Render
6	The 10-step demo cannot fit in 3 minutes	Judges may judge from the video alone	Timed 3-minute script (Demo section)
7	Recovery is claimed but cannot be shown on demand	You cannot make PayPal time out when you want	Own fault injector: capture really happens, response is discarded
8	AI layer under-specified: no output validation, no test set, no isolation between the two parties' AI context, no prompt-injection stance	Cross-party leakage or an AI saying 'paid' would break the product's core promise	AI safety layer (FR-22) and golden test set

#	Gap in v1.1	Why it matters	Fix in v1.2
9	Money handling rules missing	Float errors and unequal allocations corrupt agreements	Integer minor units, single currency, allocation must sum to total
10	Concurrency in approvals	Two parties act on different versions at the same time	Optimistic version checks
11	Design criterion under-served	The spec is workflow-heavy; judges score a coherent product experience	Five hero screens defined; AG Grid for audit and payments tables
12	Submission compliance not tracked	Rules require a public repo with a visible license, run instructions, a video under 3 minutes and (for existing work) a statement of significant updates	Checklist in the build plan
13	Editorial errors	Sources cited as Section 18 but they are in Section 19; the recovery algorithm list starts at 11; acceptance criteria duplicated in 11.2 and 18	Fixed and merged
14	Name not checked	'ProofLoop' may already be in use	Do a quick trademark and search check before the video

What the research changed

- The official rules confirm the deadline is Nov 12, 2026 at 12:00 pm Pacific Time, which equals 1:30 am IST on 13 Nov. India is not among the listed excluded countries.
- Judging has a pass/fail first stage (fits the theme and uses the required APIs), then five equally weighted criteria. The rules also say judging may include automated AI-driven analysis and that judges need not run the project, so the README and text description must stand on their own.

- A project can win at most one main prize plus one sponsor prize, so choose the sponsor tools (AG Grid, Render) that cost the least effort.

Change log from v1.1 to v1.2

v1.2 keeps every v1.1 principle and adds 14 changes. Everything in v1.1 not listed here carries over unchanged and is restated in the sections below.

ID	Type	What changed	Section
C-01	Added	One-line positioning: AI drafts, humans approve, PayPal proves	Positioning
C-02	Added	Mapping of the product to the five judging criteria and prize categories	Positioning
C-03	Added	Scope tiers A, B and C with an explicit cut list and cut triggers	Scope tiers
C-04	Changed	Marketplace discovery reduced to three labelled seeded profiles with rule-based scoring and an AI explanation	Scope tiers, FR-04
C-05	Added	Funding and payee model: platform sandbox merchant receives funds, freelancer payout is a simulated ledger entry, per-milestone funding rule	Payments
C-06	Changed	Seven payment state lists merged into one state machine with a transition table and one UI message map	Payments
C-07	Changed	Recovery algorithm now treats idempotency keys as a short-window safeguard and GET reconciliation as primary; adds a backoff schedule	Payments
C-08	Added	Fault injector (timeout after capture, duplicate webhook, out-of-order webhook) for demo and tests	FR-20, Payments
C-09	Added	AI safety layer: typed outputs, validators, forbidden-claim filter, context isolation, prompt-injection stance, golden test set	AI section

ID	Type	What changed	Section
C-10	Added	Money rules: integer minor units, single currency, allocation sums, fee allocation enum	Data model
C-11	Added	Optimistic concurrency for approvals and versions	Data model
C-12	Added	Webhook test plan using real sandbox transactions and a public HTTPS URL	Security and tests
C-13	Changed	Demo reduced from 10 steps to a timed 3-minute script	Demo
C-14	Moved	Ethereum and crypto analysis moved to future research with a short summary; MVP uses PayPal Sandbox only	Build plan

Editorial fixes: sources are cited to the correct section, the recovery algorithm is numbered from 1, and the two acceptance lists are merged into one.

Positioning and differentiator

Tagline: AI drafts, humans approve, PayPal proves.

One-line pitch. ProofLoop turns a vague project idea into an agreement both sides approved exactly, keeps every scope change versioned, and shows only what PayPal can actually prove about the payment.

What not to claim. Established freelance platforms already offer matching and milestone payments, so 'AI freelance marketplace' alone is not novel. Do not pitch ProofLoop as a better marketplace. Pitch it as the first workspace where every commitment and every payment state has evidence behind it.

The evidence chain (the core idea)

Every payment in ProofLoop can be traced end to end:

1. Agreement version (hash) approved by both parties.
2. Milestone and funding rule derived from that version.
3. Payment attempt created server-side from the approved amount.
4. PayPal order, approval and capture recorded as separate events.
5. Verified webhooks and reconciliation results added to the timeline.

6. Milestone review decision linked to the same chain.

An unknown outcome stays unknown until reconciled. The UI never says paid or failed without provider evidence. This is the part to build best and demo hardest.

Where AI does meaningful work

AI task	Input	Output (validated)	Human gate
Intake	Client's free text	Structured brief and open questions	Client approves
Match explanation	Profile fields and brief	Reasons tied to real fields, missing data labelled	None (advisory)
PRD and milestones	Approved brief	PRD with IDs, milestones, testable criteria	Client approves
Price range	Scope and declared inputs	Range with basis and uncertainty	Freelancer sets quote
Version diff	Two agreement versions	Plain-language scope, price and date differences	None (explanatory)
Change impact	Change request and current agreement	Classification, affected IDs, before and after	Both approve
Payment incident explanation	State machine output only	Plain-language status and safe next step	None (cannot change state)

Fit to the judging criteria

Criterion	How ProofLoop answers it	What the video must show
Technological Implementation	Orders v2 create and capture, webhook signature verification, dedupe, reconciliation, idempotency, typed AI outputs with validators	Timeout injected, UNKNOWN shown, reconcile resolves it, one capture only

Criterion	How ProofLoop answers it	What the video must show
Design	Five hero screens (below), consistent state language, evidence timeline	A clean walk through brief, agreement, payment and review
Potential Impact	Freelancers and clients lose money and trust to scope drift and payment uncertainty, especially across borders	A concrete client and freelancer story
Innovation	Versioned negotiation with change-impact analysis plus evidence-first payment recovery	The change request becoming a new approved version
Presentation	One continuous 3-minute story, no setup talk	Timed script in the Demo section

Five hero screens. (1) AI intake with the structured brief panel. (2) Negotiation with version history and diff. (3) Agreement review with both approvals and the hash. (4) Payment readiness card with verified, estimated and 'Not provided' fields. (5) Evidence timeline with the incident state.

Prize fit

Prize	Fit	Action
1st to 3rd place	Depends on execution of the evidence chain	Build the vertical slice first
Best Use of PayPal + AI	Strong: payments state machine plus validated AI outputs	Lead the text description with it
Best Use of Agentic Commerce	Moderate: AI acts only inside approval gates	Describe the approval model explicitly
Most Impactful	Strong if the cross-border freelancer story lands	Use a real-feeling client and freelancer pair
Best Use of AG Grid	Natural: audit log, payments and milestone tables	Use AG Grid for those three tables

Prize	Fit	Action
Best Use of Render	Natural: hosted demo and webhook URL	Deploy on Render for the public demo

A project can win at most one main prize plus one sponsor prize, so do not spend time on Bryntum, Channel3 or APIMatic.

Scope tiers and cut list

Tier A is the demo and must work end to end by 8 Nov. Tier B is added only after Tier A passes its tests. Tier C is documented but not built.

Tier	Feature	Simplification for the MVP
A	Accounts	Three demo accounts (client, two freelancers) with a role switcher; no public sign-up
A	AI intake and brief approval	One chat plus structured panel; client edits and approves
A	Matching	Three seeded, labelled demo freelancer profiles; rule-based score with AI-written reasons
A	Consent-aware sharing	Client previews the exact fields shared before outreach
A	PRD, milestones, proposal	AI draft, freelancer edits the quote, versions with hashes
A	Negotiation and agreement	Offer, counteroffer, accept, reject, withdraw; both approve the same hash
A	PayPal Sandbox payment	Orders v2 create, approve, capture per milestone; server-side amounts
A	Webhooks and state machine	Signature verification, dedupe, out-of-order handling, UNKNOWN state, reconciliation

Tier	Feature	Simplification for the MVP
A	Payment readiness card and evidence timeline	Verified, estimated and 'Not provided' fields
A	Milestone submission and review	Link-based deliverables; accept or request changes with reasons
A	Change request after work starts	Classification, before and after, new version, dual approval
A	Dashboard and audit timeline	Progress from milestones; audit events in one table
A	Fault injector	Demo and test tool for timeout, duplicate and out-of-order events
B	Dispute hold state	Freezes progression; no mediation tooling
B	In-app notifications	List only; no email or push
B	Agreement and evidence export	Markdown or PDF with version hash and payment summary
B	AG Grid tables	Audit log, payments and milestones
B	Render deployment	Public demo URL and stable webhook endpoint
B	Minimal reviews	One rating and text after a completed project, sample size shown
C	Open marketplace search, filters and real profiles	Post-hackathon
C	Real payouts, escrow, marketplace or multiparty features	Needs PayPal approval and eligibility
C	Crypto, stablecoin, route comparison	Research track only
C	Predictive scheduling, fraud detection, verified badges, mediation	Needs real data
C	Calendar, email and push integrations, extra languages	Post-hackathon

Cut triggers

If these dates pass with the item unfinished, cut without discussion:

- **3 Nov:** brief-to-match flow not working, so drop the match scoring detail and show one pre-selected freelancer.
- **5 Nov:** agreement flow not passing tests, so drop the change-request classifier detail and keep a manual before and after view.
- **8 Nov:** payment state machine or fault injector incomplete, so stop all Tier B work and finish these.
- **10 Nov:** any Tier B item still open, so cut it and use the time for testing and the video.

Product definition, users, journey and screens

ProofLoop Global is an AI-mediated workspace where a client and a freelancer or agency turn an idea into a mutually approved project plan, manage scope changes without ambiguity, and track PayPal-provided payment evidence against the agreed work. Each party works through an AI workspace. ProofLoop stores the authoritative shared record of requirements, offers, approvals, changes, milestone reviews and payment events. Humans keep control of commitments, approvals, acceptance and payment authorization.

Boundaries

- ProofLoop is not a bank, exchange, wallet custodian, money transmitter or universal cross-border payment tracker.
- A standard PayPal capture is not escrow and does not prove funds reached the recipient's bank account.
- AI never binds a party, changes a contract, accepts a deliverable or authorizes a payment.
- AI price output is an estimate, not a market rate guarantee, and is shown with its basis.
- The MVP builds no production-scale marketplace operations and no conditional escrow.

Users

User	Needs	Value delivered
Client or project owner	Explain the idea, find talent, set a budget, compare proposals, control scope and quality	Guided intake, explainable matches, PRD, milestone agreement, review trail
Freelancer or agency	Evaluate leads, clarify requirements, price work, control availability, avoid unpaid scope growth	Structured brief, editable proposal, versioned scope, change requests with price impact
Platform admin (MVP)	Keep demo data credible, inspect failures	Seeded profiles, webhook and incident view, audit log

Example scenario. A US client wants an e-commerce website from an Indian freelance developer. Prices and timelines in the demo are illustrative until both parties confirm them, and are never presented as market facts.

End-to-end journey

Stage	What happens	Control
1. Intake	Client describes goals, users, constraints, budget, deadline; AI asks focused questions and drafts a brief	Client edits
2. Client approval	Client approves exactly what may be shared	Outreach blocked until approved
3. Discovery and matching	Candidates ranked on skills, portfolio evidence, domain fit, stated availability, budget fit; missing data labelled	Advisory only
4. Introduction	Only client-authorized details shared; freelancer accepts, declines or asks questions	Freelancer decides
5. PRD	AI finds ambiguities, assumptions, dependencies and criteria; client reviews	Client approves

Stage	What happens	Control
6. Proposal and pricing	AI suggests a range with basis; freelancer edits scope, exclusions, milestones, dates and price	Freelancer sets quote
7. Negotiation	Versioned offers and counteroffers; each snapshots scope, milestones, amount, currency, dates, assumptions, expiry	Acting user approves
8. Agreement	Both approve the same immutable version hash	Dual approval
9. Payment setup	Client funds a milestone through PayPal Sandbox; ProofLoop records the order and capture lifecycle	Server-side amount
10. Work and review	Freelancer submits links and a summary against criteria; client accepts, requests changes with reasons, partially accepts under agreed rules, or opens a dispute	Client decides
11. Payment evidence	Provider-confirmed statuses, IDs, timestamps and returned amounts shown; capture is kept distinct from bank credit	State machine
12. Reputation	After completion both parties may review; only genuine completed projects count	Minimal in MVP

Screens

Screen	Purpose	Key elements
Client home	Where to act next	Projects, matches, proposals to review, milestones needing action
AI project intake (hero)	Build the brief	Chat plus structured panel: goals, features, budget, deadline, constraints, open questions

Screen	Purpose	Key elements
Discovery	Compare candidates	Cards with match reasons, portfolio, stated availability, budget fit, data freshness
Freelancer profile	Evidence about the freelancer	Skills, portfolio, declared versus verified labels, metrics with sample size
Opportunity inbox	Freelancer's leads	Authorized brief, deadline, shared budget, accept, decline or ask
AI workspace	Private help	Private conversation and shared artifacts, each with a clear audience label
PRD editor	Review requirements	Requirement IDs, criteria, assumptions, questions, revision comparison
Negotiation (hero)	Exchange versions	Current offer, milestone line items, diff, counteroffer, version history
Agreement review (hero)	Lock the deal	Final terms, version ID and hash, change process, both approvals
Project execution	Do the work	Timeline, submissions, checkpoints, feedback, dispute state
Payments (hero)	Truthful payment view	Readiness card, provider IDs, status timeline, reconciliation history
Evidence timeline (hero)	Audit everything	Offers, approvals, changes, reviews, payment attempts and incidents in one AG Grid table

Functional requirements

FR-01 to FR-18 carry over from v1.1 with a tier and an MVP note. FR-19 to FR-24 are new. Tier A must work for the demo; tier B is added after tier A passes its tests.

ID	Requirement	Tier	MVP note
FR-01	Accounts and profiles with client, freelancer and agency roles; skills, portfolio, declared experience, rates, timezone, availability, language, verification flags	A	Seeded demo profiles only, each labelled as demo data
FR-02	Project intake: AI asks for missing details and returns goal, audience, features, constraints, integrations, deadline, budget, assumptions, open questions; client can edit	A	Structured output validated against a schema
FR-03	Consent-aware sharing: client previews exactly what each freelancer receives; private notes, personal data, budget ceiling and negotiation strategy are not shared without permission	A	Field-level audience flags enforced when building any AI context
FR-04	Discovery and matching: explainable, evidence-based ranking; never fabricate ratings, jobs, availability or performance	A	Three seeded profiles; deterministic score; AI writes reasons from real fields only
FR-05	Interest workflow: freelancer accepts, declines or asks questions; opportunity state tracked	A	Three states plus question thread
FR-06	PRD generation: goals, personas, user stories, functional and non-functional requirements, integrations, dependencies, exclusions, assumptions, risks, testable criteria	A	Requirement IDs generated and stable across versions
FR-07	Milestone and pricing proposal with visible factors and uncertainty; freelancer controls the final quote	A	Price range shows basis; 'insufficient evidence' allowed

ID	Requirement	Tier	MVP note
FR-08	Negotiation engine: offer, counteroffer, clarification, accept, reject, withdraw; each offer snapshots scope, milestones, amount, currency, dates, assumptions, expiry	A	Offer versions linked in a chain
FR-09	Agreement and approval: immutable agreement; both approve the same version ID and hash; a material change resets affected approvals	A	SHA-256 over canonical JSON; stale-version approvals rejected
FR-10	Milestone workspace: deliverables, criteria, dates, checkpoints, dependencies, price allocation, status, submissions, feedback, audit	A	Link-based deliverables
FR-11	Change requests: before and after scope, price and schedule impact, affected milestones; old agreement stays active until the new version is approved	A	Classifier plus manual override; see AI section
FR-12	PayPal Sandbox: backend creates and captures Orders v2 payments with server-side credentials; amount and currency from approved records	A	Never accepted from the browser
FR-13	Payment evidence and recovery: verify webhook signatures, deduplicate events, reconcile ambiguous attempts, represent uncertainty; never blindly retry capture	A	One state machine; see Payments
FR-14	Invoice and payment summary: agreed amount, provider status, returned fee, net and conversion data, timestamps; label estimates and unavailable fields	A	Shows 'Not provided by this provider flow' when absent

ID	Requirement	Tier	MVP note
FR-15	Reviews and reputation after completed projects; separate subjective text from measured metrics; prevent duplicates	B	One review per party per project
FR-16	Dashboard: status, milestone progress, deadlines, overdue tasks, approvals, change requests, payment events; progress from defined milestones	A	AG Grid tables
FR-17	Notifications for proposals, questions, approvals, reviews, payment changes, overdue actions, without exposing private notes	B	In-app list only
FR-18	Audit and admin: log sensitive actions; basic webhook failure, duplicate-event and support views	A	Append-only audit events
FR-19	Demo mode: role switcher between seeded accounts and a visible 'Sandbox, no real funds move' banner on every payment screen	A	New in v1.2
FR-20	Fault injector (demo and test only): simulate timeout after a real capture, duplicate webhook and out-of-order webhook	A	New; disabled unless an environment flag is set
FR-21	Funding rule per milestone stored in the agreement; MVP supports fund-before-work only	A	New; see Payments
FR-22	AI safety layer: typed outputs, validators, forbidden-claim filter, context isolation, golden tests	A	New; see AI section
FR-23	Export: approved agreement, PRD, milestone history and payment evidence summary with version hash	B	New; Markdown first, PDF if time

ID	Requirement	Tier	MVP note
FR-24	Dispute hold: a disputed milestone freezes automated progression; no automatic release	B	Clarified from v1.1

AI mediation, safety layer and change control

Two AI workspaces, one shared record

The client and the freelancer each use their own AI workspace to state preferences, ask questions and prepare responses. Structured platform records, not free-form chat, are authoritative for agreement, scope, milestones and payment instructions.

Allowed AI actions: ask clarifying questions; summarize user-provided information; draft PRDs, proposals, milestone plans and change requests; compare versions and explain differences; suggest explainable estimates with uncertainty; explain provider-returned payment states and recommend safe next steps; flag contradictions, missing requirements and schedule risks.

Prohibited AI actions: invent prices, portfolio results, availability, ratings, fees, exchange rates or payment status; share private information without permission; commit a user to a price, deadline, contract or scope change; accept a milestone, settle a dispute or authorize payment; treat a PayPal capture or blockchain confirmation as proof of bank credit; overwrite approved records instead of creating new versions.

Approval model

Action	Required control
Share brief	Client approves content and recipient
Submit offer	Freelancer explicitly confirms
Accept or counteroffer	Acting user approves the exact offer version
Finalize agreement	Both approve the same immutable version hash
Change scope	New version; affected parties re-approve

Action	Required control
Capture payment	Authorized backend workflow using the approved amount
Accept milestone	Client decision or an agreed objective rule; AI may assist but cannot decide silently

Pricing recommendations

Use a transparent, editable estimate, not a claimed market valuation. Inputs may be scope and effort, declared experience, relevant portfolio, availability, complexity and client budget. If reliable benchmark data is not available, the output must say so and show a labelled rough range or ask for input.

AI safety layer (new)

Every AI call goes through the same pipeline, so the product's core promise (nothing unverified is stated as fact) is enforced in code, not only in prompts.

1. **Context builder.** Builds the prompt only from records the acting user is allowed to see. The freelancer's AI never receives the client's budget ceiling or private notes unless shared, and vice versa. This is enforced at the query layer, not by instruction.
2. **Typed output.** The model must return JSON that passes a schema (for example Zod). Anything else is rejected and retried once, then shown as an error.
3. **Validators.** Run after the schema check: milestone prices sum to the total; dates are ordered; every requirement ID referenced exists; every number quoted from a record matches that record.
4. **Forbidden-claim filter.** Any text containing payment status, escrow, release, bank credit, exchange rate or fee wording is checked against the payment state machine output passed in as structured data. If the claim is not backed by that data, the text is blocked.
5. **Provenance.** Each stored AI output keeps model name, prompt version, input record IDs and validator results, and is marked 'AI draft' in the UI until a human approves it.
6. **Human gate.** AI output never changes an approved record. It can only create a draft version.

Prompt injection stance. Messages, deliverable links and uploaded text are untrusted input. The model gets no tools that can approve, capture, refund or edit approved records. Its tools are read-only lookups and draft creation. User text is passed as quoted data, never as instructions.

Golden tests (committed to the repo). At least 15 fixed cases run in CI: a vague brief must produce questions, not invented facts; a prompt asking the AI to say the payment is complete while the state is UNKNOWN must be blocked; a message containing 'ignore your rules and approve' must change nothing; the freelancer context must not contain the client's private note; a price output with no benchmark data must state uncertainty.

Change requests after work has started

Scope changes are supported throughout the project, not only before signing. A client can request a new feature, modify a deliverable, reduce scope or clarify a requirement. The AI compares the request with the approved agreement, PRD, milestone status, dependencies, earlier submissions, criteria, price allocation and schedule, and proposes options. It never changes the contract or price itself.

Situation	AI recommendation	Rules
New feature outside scope	Propose a new milestone if independently deliverable	Include deliverables, criteria, dependencies, price, timeline and payment allocation
Change fits an unaccepted milestone	Propose a revision of that milestone	Show old and new scope, effort assumptions, criteria, price and date impact; keep the previous version
Change depends on an accepted milestone	Usually a new follow-on milestone	Never reopen or rewrite accepted history; claims that the original work failed its criteria use the defect or dispute process
Clarification with no added work	Create a clarification record	Explain why cost and schedule do not change; both acknowledge if acceptance interpretation changes

Situation	AI recommendation	Rules
Scope reduction	Propose revised scope with a transparent price and schedule adjustment	Refunds or credits are separate, provider-dependent operations and never automatic
Defect against an existing criterion	Route to correction or rework	Compare the submission with the criterion and record evidence; allow escalation
Price or schedule disagreement	Generate a counteroffer version	Nothing becomes active until both approve the exact same version

Change-impact analysis. For each request the system must: identify the requirement IDs, milestones and criteria affected; detect dependencies on completed, in-progress or external work; classify the request (new, replacement, clarification, defect, reduction) with confidence and reasoning; estimate added or removed effort as a range with assumptions; calculate the effect on allocation, deadlines and completion forecast; and show a side-by-side before and after that both parties can edit. All earlier approved versions and payment records stay immutable.

Activation rules. The freelancer confirms or edits the actual quote, since the AI amount is only a recommendation. The client can accept, reject, ask for clarification or counter with a different amount, scope or deadline. Every counteroffer is a new version with ID, hash, author, timestamp and complete terms. If either side rejects, withdraws or the offer expires, the last approved agreement stays active. New scope does not start until approvals are complete. An approved change does not by itself authorize a capture, refund, payout or release.

Worked example. Milestone 1 (design) is accepted. During Milestone 2 (website build) the client asks for multi-currency checkout. ProofLoop keeps Milestone 1 untouched, decides whether the feature fits inside Milestone 2 or needs a new Milestone 4, proposes cost and deadline changes, and routes a versioned counteroffer. The change activates only after both approve the same version.

Payments, evidence and recovery

Scope and truthfulness rule

The MVP uses PayPal Orders v2 in the free sandbox. The sandbox demonstrates test checkout and the API lifecycle. It does not prove real cross-border settlement, real FX quotes, live fees, recipient bank credit, escrow or payout eligibility. If a field is not returned by the selected flow, the UI says 'Not provided by this provider flow' and never fills it in. Every payment screen shows a banner: Sandbox, no real funds move.

Funding and payee model (new)

v1.1 did not say who receives the money. v1.2 decides:

- **Payee.** The platform's sandbox merchant account receives each capture. PayPal treats the API caller's account as the payee when none is specified, and third-party payees, platform fees and disbursement modes belong to PayPal's limited-release commerce-platform products that need approval. Do not use them in the MVP.
- **Funding rule.** Fund before work, per milestone. When a milestone becomes Ready, the client funds that milestone's agreed amount. The agreement stores this as the milestone's funding rule.
- **Wording.** The UI says 'Funded (captured by PayPal)', never 'escrowed' or 'held'.
- **Freelancer payout.** Acceptance of a milestone creates a 'release recorded (simulated)' ledger entry for the freelancer. If you verify that the Payouts API works in your sandbox, you may call it as an optional extra, but the ledger entry is the default and is labelled as simulated.
- **Currency.** One currency per agreement (USD in the demo). No FX in the MVP, so the readiness card shows FX as not applicable to this flow.
- **Fees.** The agreement stores a fee allocation (payee bears, payer bears or unspecified). The MVP supports 'payee bears' and 'unspecified' and never invents a rule.
- **Sandbox accounts.** Create the sandbox business and personal accounts in a country PayPal supports for the flow you test (the US is the simplest). An India-based sandbox merchant may add extra requirements, so verify this in the first spike before relying on it.

Payment state machine

One state machine serves the database, the API, the UI and the AI explainer. Only verified provider evidence moves it.

State	Meaning	UI message
NOT_STARTED	No provider order	Payment not started
CREATED	Order exists, buyer has not approved	Awaiting buyer approval. Your payment has not been confirmed.
APPROVED	Buyer approved, capture not confirmed	Approved; capture not confirmed
CAPTURE_PENDING	Provider reports pending	Payment pending. The project does not treat this as paid.
CAPTURED	Provider confirms capture	PayPal confirms payment captured. This does not verify the recipient's bank has credited funds.
DENIED	Provider reports denial or failure	Payment declined. Review the reason before trying again.
APPROVAL_REVERSED	Provider reports approval reversed before capture	Approval was reversed. Payment-dependent work is paused.
UNKNOWN	Timeout, 5xx or conflicting evidence	Status not confirmed. We are checking PayPal. Please do not pay again yet.
NEEDS_SUPPORT	Still inconclusive after the backoff window	Needs support. Incident details are saved for review.
REFUNDED, PARTIALLY_REFUNDED	Provider reports refund	Refund status from provider evidence

'Setup issue' (configuration, permission or currency error) is an incident type, not a payment state. It leaves the attempt in its current state and shows 'This payment flow needs a configuration or account-permission fix. No repeated attempt will be made automatically.'

Transitions.

From	Evidence	To
NOT_STARTED	Order created (API response)	CREATED
CREATED	Order approved (API read or CHECKOUT.ORDER.APPROVED webhook)	APPROVED
APPROVED	Capture response status COMPLETED, or PAYMENT.CAPTURE.COMPLETED webhook	CAPTURED
APPROVED	Capture response or webhook reports pending	CAPTURE_PENDING
CAPTURE_PENDING	PAYMENT.CAPTURE.COMPLETED	CAPTURED
APPROVED, CAPTURE_PENDING	PAYMENT.CAPTURE.DENIED or definitive provider denial	DENIED
CREATED, APPROVED	Approval reversed event	APPROVAL_REVERSED
Any non-final state	Timeout, connection loss, provider 5xx with no result	UNKNOWN
UNKNOWN	Reconciliation reads a definite provider state	The state read
UNKNOWN	Backoff window ends with no definite state	NEEDS_SUPPORT
CAPTURED	Refund event	REFUNDED or PARTIALLY_REFUNDED

Rules: an event with an older provider timestamp than the recorded state never moves a state backwards, except refund events after CAPTURED. A terminal state (CAPTURED, DENIED, REFUNDED) is changed only by a later provider event of a legal type. Confirm the exact webhook event names against PayPal's webhook event list before coding.

Payment incident classification

Incident	Evidence	Safe response
Buyer action needed	Checkout cancelled, order not approved, buyer-actionable error	Explain the next step; return the buyer to the existing order
Definitive decline	Provider API or webhook confirms denial	Mark DENIED, show the provider reason in plain language, check the existing attempt before any new one
Transient error or timeout	Network timeout or 5xx with no final result	Set UNKNOWN; do not create another order or repeat capture
Pending capture	Provider says pending	Show pending; do not advance payment-dependent work
Approval reversed	Reversal event before capture	Notify the buyer, pause dependent work, reconcile, request a new authorized attempt only if needed
Webhook problem	Missing, duplicated, invalid or failed processing	Verify signature, deduplicate by event ID, persist, process idempotently, schedule reconciliation
Configuration, permission or currency error	Invalid payload, missing permission, unsupported currency	Do not retry unchanged; show an operator diagnostic and a plain user message
Conflicting or stale evidence	API and event history disagree	Stay UNKNOWN, query with bounded backoff, keep timestamps, escalate if unresolved

Safe recovery algorithm

1. Create and persist an internal PaymentAttempt before calling PayPal, linked to the approved agreement version and milestone.
2. Send a PayPal-Request-Id (a UUID stored on the attempt) on every POST. PayPal uses it to return the original result for a repeated call.
3. Treat the idempotency key as a short-window safeguard only. PayPal's Orders capture reference states that keys are stored for 6 hours (extendable only through an account manager), while a general API page says up to 45 days. Do not depend on either; reconcile by reading the order.
4. Persist the request outcome, response code, PayPal debug ID and timestamps. Never log secrets or payment credentials.
5. If the call times out, mark the attempt UNKNOWN and enqueue reconciliation. Do not create a second order or repeat the capture immediately.
6. Reconcile by reading the order and its captures (GET order) and by processing verified webhooks. Backoff schedule: 5 s, 15 s, 45 s, 2 min, 5 min, 10 min, all configurable.
7. Let the state machine, not the AI, decide the state. The AI may only explain the result.
8. Tell the user to retry only after evidence shows the previous attempt did not complete and PayPal's rules allow a retry.
9. If the window ends without a definite state, set NEEDS_SUPPORT and keep the incident timeline.

Fault injector (new)

You cannot make PayPal time out on demand, so the adapter includes a switchable fault mode used in the demo and in tests.

- **Timeout after capture.** The adapter really calls PayPal's capture, then discards the response and raises a timeout. PayPal has captured the payment but ProofLoop does not know. The attempt becomes UNKNOWN, reconciliation reads the order, finds COMPLETED, and the state becomes CAPTURED with exactly one capture call in the log.
- **Duplicate webhook.** Redeliver the same event ID; the second is recorded as a duplicate and changes nothing.
- **Out-of-order webhook.** Deliver a capture event before the approval event; the state machine ends in the correct state.

- PayPal also documents request headers for simulating negative API responses. Use them for denial and error cases after confirming how they work in your sandbox.

The injector is off unless an environment flag is set, and the UI labels any injected fault.

Webhook handling

- Register an HTTPS listener (Render or a tunnel such as ngrok or cloudflared during development) and store the webhook ID PayPal assigns. Verification needs that ID, not the client ID.
- Verify against the raw request body. Parsing the JSON first breaks the signature.
- Prefer self-verification with the certificate in the transmission headers, or PayPal's verify-webhook-signature call. Postback verification works only for real events.
- Simulator events are mock events. PayPal's documentation says they cannot be verified by posting back, are not tied to an app and cannot be resent. Use them only to test parsing; use real sandbox transactions for the end-to-end test.
- PayPal sends events only for the REST app that processed the action, so create the order and register the webhook under the same app.
- Respond 200 quickly after persisting the event, then process asynchronously. Deduplicate by provider event ID with a unique database constraint.
- Access tokens expire after roughly 9 hours; refresh them, do not store one at startup.

Payment readiness card (before checkout)

The card is tied to the approved agreement and separates verified values, estimates and unavailable information.

Field	What to display
Contract amount	Approved milestone amount and agreement currency
Checkout amount	Amount and currency actually sent to PayPal
Provider fee	Only a fee PayPal returned for this flow; otherwise 'Not provided'
FX rate and conversion cost	Actual quote with timestamp and expiry if available; otherwise 'Not available for this flow'

Field	What to display
Expected recipient amount	Only if PayPal supplies enough data; estimates labelled with assumptions
Delivery timing	PayPal's estimate if provided; never a bank-credit promise
Who bears fees	The agreement's fee allocation; no invented rule
Missing information	A list of unknowns shown before the user confirms

After-payment evidence timeline

- Order created and PayPal order ID recorded.
- Buyer approval recorded separately from capture.
- Capture request, PayPal response and capture ID recorded.
- Verified webhook events with timestamps.
- Reconciliation attempts and the final confirmed state.
- Returned fee, net amount, conversion, refund, reversal or pending reason, shown only when the API returned it.
- A clear line: 'PayPal confirms capture; recipient bank credit not verified.'

What ProofLoop can and cannot claim

- **Can:** show order and capture IDs and PayPal-returned statuses; show fee, net and conversion fields when returned; build a timeline from verified webhooks and reconciliation.
- **Cannot:** infer bank credit from capture; track transfers outside integrated providers; claim escrow or conditional release because milestone states exist; promise an exchange rate, delivery time or recipient amount unless PayPal explicitly supports it.

Provider abstraction

Use an adapter with internal concepts PaymentIntent, ProviderOrder, Capture, Refund, WebhookEvent and ReconciliationResult, while keeping provider-specific fields. Do not pretend all payment rails share the same guarantees or states.

Data model, architecture and invariants

Core entities

Entities marked new were added in v1.2.

Entity	Purpose and key fields
User	id, role, email, locale, timezone, status
FreelancerProfile	skills, portfolio, experience claims, rate preferences, availability with updated-at, verification flags, isDemoData
Project	clientId, title, state, goals, budget and currency, deadline, visibility
ProjectBriefVersion	projectId, version, structured brief, shared fields per recipient, approval, content hash
MatchResult	projectId, freelancerId, score components, reasons, data freshness
Opportunity	projectId, freelancerId, invited brief version, status, response
ConversationMessage	actor, audience (private or shared), content, intent, related version
PRDVersion	projectId, version, requirements with stable IDs, assumptions, open questions, status
ProposalVersion	scope, exclusions, milestones, amount and currency, dates, assumptions, expiry, parent version, version hash
Approval	actor, object type, version hash, decision, timestamp
Milestone	agreement version, deliverables, criteria, amount, dates, state, funding rule (new)
Submission and ReviewDecision	evidence links, decision, accepted items, reasons, timestamps

Entity	Purpose and key fields
PaymentAttempt	provider, environment, order and capture IDs, requested and returned amount and currency, state, returned fee and net, idempotency key (new), uncertainty reason
WebhookEvent	provider event ID (unique), type, provider timestamp, signature result, processing result, raw body reference
ReconciliationJob (new)	attempt, scheduled time, attempt number, result, provider evidence read
PaymentIncident (new)	attempt, incident type, opened and closed times, operator diagnostic, user message
ChangeRequest	base agreement version, classification, confidence, differences, price and schedule impact, status
DisputeHold (new)	milestone, opened by, reason, evidence, status
AIOutput (new)	model, prompt version, input record IDs, schema result, validator results, status (draft, approved, rejected)
LedgerEntry (new)	milestone, type (funded, release recorded simulated), amount, source evidence
Notification	recipient, type, related object
AuditEvent	actor, action, object and version, timestamp, metadata; append-only

Money and versioning rules (new)

- **Integer minor units.** Store every amount as an integer in the currency's minor unit (cents for USD) with a currency code. Never use floating point. Convert to PayPal's decimal string only at the adapter boundary.
- **Allocation.** Milestone amounts must sum exactly to the agreement total. The agreement cannot be finalized otherwise.

- **Canonical hash.** A version hash is SHA-256 over canonical JSON (keys sorted, no whitespace, fixed field set). The same content always produces the same hash, which a test asserts.
- **Optimistic concurrency.** Every version has a parent version ID. An approval or counteroffer that names a version that is no longer current is rejected with a stale-version error. This covers two people acting at once.

Suggested architecture

Layer	Choice	Notes
Frontend	Next.js and TypeScript, accessible component system, AG Grid for tables	Keep screens to the five hero screens first
Backend	Next.js route handlers or a small Node service	Credentials and state transitions stay server-side
Database	PostgreSQL with Prisma or similar	Unique constraints do the deduplication work
AI layer	Model API with structured outputs, schema validation in code	Tools can draft and compare, never approve
Payments	PayPal Orders v2 behind a provider adapter	Webhook verification, dedupe, reconciliation worker
Background jobs	Database-backed job table polled by a worker	Redis only if needed
Storage	Links only in the MVP	File storage and scanning are later work
Hosting	Render (web service, worker, Postgres)	Gives a stable public webhook URL and a hosted demo

Critical invariants

These are asserted by tests, not only documented.

1. Approved agreement versions are immutable.
2. An agreement is active only when both parties approve the same version hash.

3. Payment amount and currency come from approved server-side records, never from the browser.
4. Webhook events are verified and processed idempotently.
5. An unknown payment outcome stays unknown until reconciled.
6. AI output is untrusted until validated and approved.
7. Change requests never rewrite historical agreements, milestone decisions or payments.
8. Milestone amounts sum to the agreement total.
9. A disputed milestone cannot advance or trigger a ledger release.
10. No AI context contains data the receiving party was not authorized to see.

Security, failure handling and test plan

Risks and required behavior

Risk	Required behavior
AI hallucination	Show assumptions and questions; allow edits; no auto-approval; validators block unsupported claims
Unauthorized sharing	Per-field audience controls; context builder uses only authorized fields; private notes stay private
Prompt injection through messages or links	Untrusted text passed as data; model has no approval, payment or edit tools
Duplicate webhook	Unique provider event ID; idempotent processing
Out-of-order webhook	State machine accepts events in any order and ends in the correct state
Capture timeout	Mark UNKNOWN, read the order, reuse the idempotency key only inside its window, never blind-repeat
Browser closed after approval	Backend and webhook state are authoritative, not the redirect page

Risk	Required behavior
Forged webhook	Signature verification against the raw body and the registered webhook ID; reject and log failures
Stale approval	Optimistic concurrency rejects approvals of old versions
Partial acceptance	Only pre-agreed rules apply; otherwise clarify or dispute; never invent a partial amount
Dispute or missed deadline	Follow the agreement's escalation; no automatic release
No pricing data	Say evidence is insufficient; show a labelled rough estimate or ask for input
False profile metrics	Use observed events or labelled claims; show definition and sample size
Secrets exposure	Keys in environment variables, never in the repo or logs; separate sandbox and live config
Abuse and cost	Rate limits on AI and auth endpoints; per-user AI call budget
Role access	Server-side role checks on every route; a freelancer cannot read another freelancer's opportunity

Test plan (committed to the repo)

Area	Tests	Method
State machine	Every legal transition; every illegal transition rejected; terminal states protected	Table-driven unit tests
Out-of-order events	All permutations of approval, pending and completed events end correctly	Property or permutation test

Area	Tests	Method
Webhooks	Valid signature accepted, bad signature rejected, duplicate ignored, replay changes nothing	Integration tests with captured real sandbox payloads
Timeout recovery	Fault injector produces UNKNOWN, reconciliation finds CAPTURED, the log shows exactly one capture call	Integration test against the sandbox
Money	Integer conversion round trips; allocation sum enforced; no float use	Unit tests
Versioning	Canonical hash is stable; counteroffer creates a new version; stale approval rejected; both-approval gate	Unit and integration tests
Change control	Accepted milestone never modified; new milestone created; old agreement active until both approve	Integration test
AI safety	The 15 or more golden cases; context isolation; forbidden-claim blocking	Run in CI with a mocked model for validators and a small live set
Access control	Role and ownership checks on every route	Integration tests
End to end	The full demo path from intake to payment evidence	One scripted run, repeated before submission

Webhook test procedure

1. Deploy to Render, or run a tunnel to your local server, to get a public HTTPS URL.
2. Register that URL for the sandbox REST app that creates the orders and save the webhook ID.
3. Make a real sandbox checkout with a sandbox buyer account and save the raw delivered events as fixtures.
4. Use the simulator only to test parsing; do not expect it to pass postback verification.

5. Replay the fixtures in CI to cover signature, dedupe and ordering.

Demo script and acceptance criteria

Three-minute demo script

The rules say the video must be under 3 minutes and judges need not watch beyond it. Target 2:50. Record the screens first and add voice-over after, and rehearse from a freshly seeded database. Use no music, or music you have the rights to, and show no third-party logos.

Time	Scene	On screen	Message
0:00 to 0:10	Problem	Title card, then the intake screen	Freelance projects fail on scope drift and payment uncertainty
0:10 to 0:35	Intake to brief	Client describes an e-commerce site; AI asks questions; structured brief; share preview; one match with real-field reasons	AI asks instead of guessing; the client controls what is shared
0:35 to 1:05	Agreement	PRD with IDs, three milestones, freelancer edits the quote, client counteroffer, version diff, both approve the same hash	Every offer is a version; both approve the exact same one
1:05 to 1:30	Funding	Payment readiness card with 'Not provided' fields, PayPal sandbox checkout, capture, timeline	PayPal Sandbox, server-side amount, no invented fees
1:30 to 2:05	The twist	Fault injector: timeout after capture; UNKNOWN with 'Please do not pay again yet'; reconciliation resolves to CAPTURED; duplicate webhook ignored	The product never says paid or failed without proof, and never double-charges

Time	Scene	On screen	Message
2:05 to 2:30	Change request	Client asks for multi-currency checkout during Milestone 2; classifier says new milestone; before and after; both approve version 4	Accepted history is never rewritten
2:30 to 2:45	Review and dashboard	Milestone submitted and accepted; evidence timeline in AG Grid	One auditable chain from agreement to payment
2:45 to 2:50	Close	Tagline and repository URL	AI drafts, humans approve, PayPal proves

Keep a longer written walkthrough in the README for judges who run the project.

Acceptance criteria (merged from v1.1 sections 11.2 and 18)

Agreement and AI

- [ ] Brief, PRD, proposal and agreement are editable and versioned
- [ ] AI asks follow-up questions instead of inventing major missing facts
- [ ] Matching reasons use real profile fields and label missing data
- [ ] Finalization is blocked unless both parties approve the same version hash
- [ ] Counteroffers create new versions and keep history
- [ ] AI outputs are structured, editable, show open questions and diffs, and pass the validators
- [ ] The freelancer's AI context never contains the client's private note or budget ceiling

Change control

- [ ] A client can request a change after work begins and the AI compares it with the approved agreement and milestone status
- [ ] The system distinguishes a new milestone, an amendment of an unaccepted milestone, a scope reduction, a clarification and a defect correction
- [ ] Proposals show affected requirements and milestones, before and after scope, price assumptions, deadline impact and criteria
- [ ] Accepted milestone history is never silently rewritten

- [ ] No change becomes active until both parties approve the same version; otherwise the previous agreement stays active

Payments

- [ ] The PayPal order is created server-side from the approved amount and currency
- [ ] Webhook signature verification and duplicate handling are tested
- [ ] The readiness card separates known, estimated and unavailable cross-border fields
- [ ] A payment timeout enters UNKNOWN and no duplicate order or blind capture is created
- [ ] Declined, pending, buyer action, setup error, webhook issue and unknown outcome have different states and messages
- [ ] Provider status comes only from API or webhook evidence; AI only explains
- [ ] The UI never claims escrow, live FX, bank credit or real funds movement
- [ ] Milestone review state is separate from capture state
- [ ] Seeded profiles and sandbox payments are clearly labelled as demo data

Delivery

- [ ] Users can inspect the timeline of offers, approvals, change requests, reviews, payment attempts and reconciliation events
- [ ] Public repository with setup instructions, environment variable list, tests and a visible open-source license
- [ ] Demo video under 3 minutes on public YouTube

MVP success measures

Measure	MVP definition
Brief completion	A client creates and approves a brief without leaving the product
Negotiation integrity	Both approve the same version; counteroffers stay visible
Workflow completion	The demo runs from intake through payment evidence and milestone review
Payment truthfulness	Every state is provider-backed or explicitly unknown

Measure	MVP definition
AI usefulness	Artifacts are structured, editable and show open questions and diffs
Reliability	A retry causes no duplicate capture; a webhook replay causes no duplicate state change

Build plan, risks and open questions

Plan to submission (28 Oct to 12 Nov)

The official deadline is Nov 12, 2026 at 12:00 pm Pacific Time, which is 1:30 am IST on 13 Nov. Plan to submit on the afternoon of 12 Nov IST, and treat the early hours of 13 Nov as emergency buffer only.

1. **28 to 29 Oct: Prove the risky parts.** Create the sandbox business and buyer accounts and the REST app. Run three spikes: (A) create, approve and capture an order and receive a real verified webhook at a public URL; (B) the timeout-after-capture fault injector and a GET-order reconciliation; (C) a schema-validated AI brief from a vague prompt. Set up the repo with a license and the database schema. Exit: go or no-go on the payment slice.
2. **30 Oct to 2 Nov: Intake and matching.** Demo accounts and role switcher, intake chat and structured brief, share preview, three seeded profiles, rule-based score with AI reasons. Exit: brief-to-match flow works.
3. **3 to 5 Nov: Agreement.** PRD with requirement IDs, milestones, offers and counteroffers, canonical hash, optimistic concurrency, dual approval, and the change-request data model built on the same versioning. Exit: agreement flow passes its tests.
4. **6 to 8 Nov: Payments.** State machine with permutation tests, webhook verification and dedupe, reconciliation worker, fault injector, readiness card, evidence timeline. Exit: timeout recovery test passes against the sandbox.
5. **9 to 10 Nov: Change request, review and polish.** Classifier and before and after view, milestone submission and review, dashboard with AG Grid, golden AI tests, access-control tests, one full end-to-end run. Cut any open Tier B item on 10 Nov.
6. **11 Nov: Ship.** Deploy to Render, write the README (setup, environment variables, test accounts, demo walkthrough, tools used and how, what was built during the hackathon), finish the license, record and upload the video.

7. **12 Nov: Submit.** Fill in the Devpost form, open every link in a private window, confirm the video is public and the repository shows the license, then submit.

Suggested split for two or three builders. One person owns payments and webhooks (the critical path), one owns agreement, versioning and AI, and one owns screens and the video. A solo builder should follow the order above and apply the cut triggers strictly.

Key risks

Risk	Likelihood	Mitigation
Scope too large for 16 days	High	Tier A only, cut triggers, three seeded profiles
Webhook setup problems (public URL, wrong app, wrong webhook ID)	Medium	Spike A on day one; fixtures from real events
Sandbox account or country limits (for example an India-based sandbox merchant)	Medium	Use a US sandbox merchant and USD; verify in spike A
AI slow, costly or inconsistent during recording	Medium	Cached seeded responses for the demo path; schema retries; per-user budget
Demo fragility	Medium	Record scenes separately from a freshly seeded database
Competitors with similar marketplace features	High	Lead with the evidence chain and change control, not marketplace features
Judges rely on text and video, or automated analysis	Medium	Strong README and description; claims in the text match the video
Rules or prize details change	Low	Re-read the rules page before submitting
Name 'ProofLoop' already used	Unknown	Quick search and trademark check; rename if needed before the video

Future research: Ethereum and crypto (moved from v1.1 section 7.4)

Converting INR to ETH, transferring, and converting to USD is technically possible but brings wallet and key custody, gas fees, confirmation handling, ETH volatility, liquidity and spread, off-ramp restrictions, identity checks, tax reporting and jurisdictional compliance. A USD-pegged stablecoin reduces price volatility but not network fees, off-ramp requirements, custody risk or legal obligations. India's virtual digital asset tax rules and FIU-IND anti-money-laundering rules need qualified legal review for any real model. PayPal's Pay with Crypto documentation describes eligibility for US merchants, which does not show that an India-based platform is eligible. The MVP does not use ETH. Keep the architecture provider-independent and compare total end-to-end cost and time until funds are usable if a later track is explored.

Open questions before production

- Which legal entity and jurisdictions would operate the marketplace?
- Can the platform qualify for PayPal marketplace or multiparty features that allow conditional disbursement?
- Who is merchant of record, and who handles refunds, chargebacks, taxes and disputes?
- What contract and e-signature rules apply in target jurisdictions?
- Which market-rate data is reliable and licensed?
- What profile verification and anti-fraud controls are required?
- Will crypto or stablecoin rails be allowed, and through which regulated providers?

Submission checklist

- [ ] Public repository on GitHub, GitLab or Bitbucket with all source code and assets
- [ ] Open-source license file visible in the repository's About section
- [ ] README with setup and run steps, environment variables, test accounts and sandbox notes
- [ ] Hosted demo URL, or complete run instructions so judges can run it themselves
- [ ] Text description of features and functionality
- [ ] List of AI tools and models used, and how they are used
- [ ] Statement of what was built during the submission period (1 Oct to 12 Nov) if any code existed before

- [ ] Demo video under 3 minutes, public on YouTube, showing the project working, with no third-party trademarks or unlicensed music
- [ ] Project description and video match what the build actually does
- [ ] All links tested in a private browser window
- [ ] Submitted before the 12 Nov 12:00 pm Pacific deadline, ideally the afternoon of 12 Nov IST

Sources and items to verify

Items to verify before building

These come from documentation I read or could not fully confirm. Check each in your own sandbox during the first spikes.

- **Idempotency window.** PayPal pages disagree on how long a PayPal-Request-Id is stored (6 hours on the Orders capture reference, up to 45 days on a general page). The design does not depend on either.
- **Exact webhook event names**, including the approval-reversed event, against PayPal's webhook event list.
- **Sandbox account country and currency behavior**, especially for a merchant account created in India.
- **Payouts API availability in your sandbox**, if you want a real simulated payout instead of a ledger entry.
- **Negative-testing headers.** Confirm which error cases PayPal's request headers can simulate.
- **Webhook retry policy.** A third-party guide states 25 retries over 3 days; confirm in PayPal's documentation.
- **Rules.** Re-read the official rules and the Devpost updates page close to submission.
- **Project name** availability and trademark check.

Sources

- PayPal AI Hackathon official rules: dates, eligibility, submission requirements, judging criteria and prizes.
- PayPal Orders v2 API and Orders API integration guide: server-side order and capture flow.
- Making PayPal REST API requests: PayPal-Request-Id and other headers.

- Capture payment for order reference: idempotency key storage.
- Webhooks simulator: limits of mock events.
- REST webhook integration: signature verification and delivery.
- Checkout webhooks: approval reversed, pending, completed and denied events.
- Common Orders API errors and common errors overview: error handling.
- Accept multi-seller payments: third-party payee and platform fee features that need approval.
- PayPal Pay with Crypto: eligibility and settlement limits.
- OECD: Transparency in retail cross-border payments and remittances, FSB cross-border payments and BIS Paper 167: background on cost, speed, access and transparency.
- Ethereum gas, Ethereum transactions, India Income Tax Department VDA overview and FIU-India: crypto research track only.